GUIDE D'APPRENTISSAGE · VERSION 0.3

Rempart, expliqué en images

Ce que fait l'outil, comment il raisonne, et pourquoi chaque morceau existe. Une idée par page, des exemples réels tirés de tes propres projets.

Code · Secrets · Dépendances · Images · AWS, GCP, Azure en direct · Kubernetes · CI/CD · Données · Agents IA · Web

316 règles de détection

16 domaines couverts

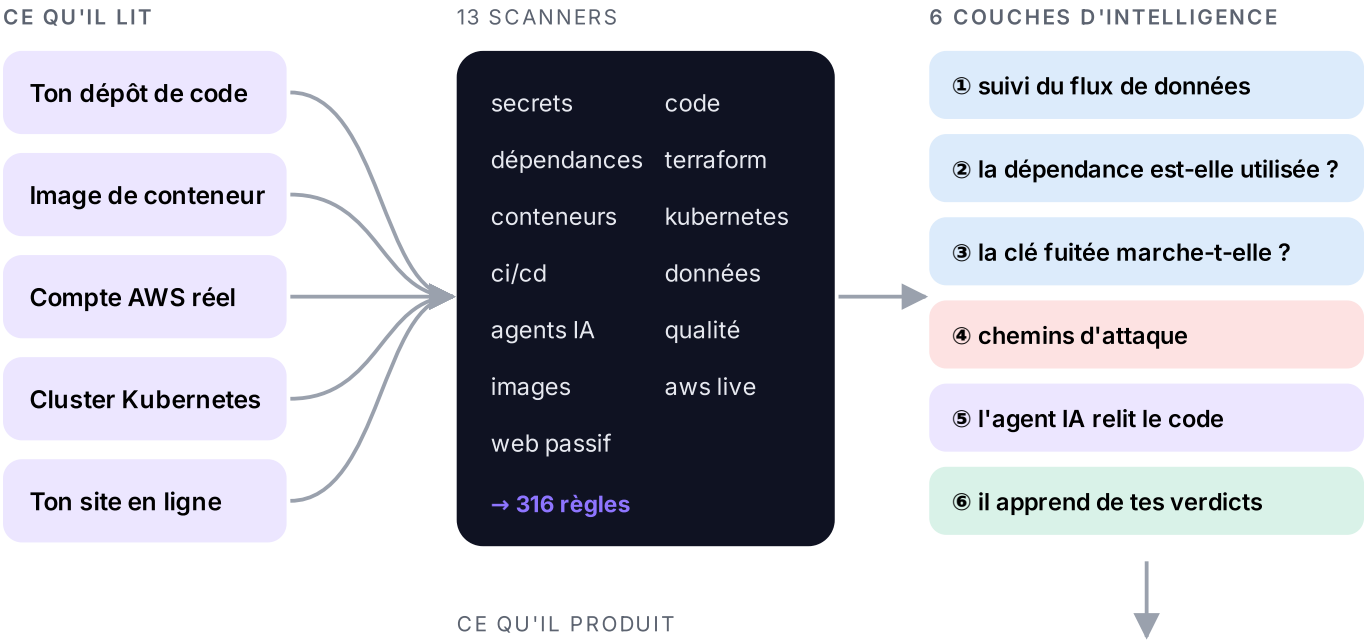
99,4 sur l'OWASP Benchmark Python

116 tests automatiques, tous verts

LA VUE D'ENSEMBLE

Une seule passe, cinq sources, une seule réponse

Rempart lit ce que tu lui donnes, cherche les faiblesses, puis les fait passer par six couches d'intelligence avant de produire un rapport.



Le principe clé

Chaque résultat dit comment il a été prouvé :

static · reachable
observed · live-secret
correlated · ai-confirmed

CE QU'IL PRODUIT

Terminal coloré	Rapport HTML	SARIF → GitHub
JSON / Markdown	Note de A à F	Outil MCP pour IA

LE MOTEUR

Un scan, étape par étape

Ce qui se passe quand tu tapes `rempart scan .` — en 0,5 à 3 secondes sur un vrai projet.

1. **Trouver les fichiers**
   Il demande la liste à Git. Ce qui est dans `.gitignore` n'est jamais envoyé sur GitHub, donc il ne le signale pas : ton `.env` local n'est pas une fuite, un `.env` committé en est une.

2. **Lancer les 13 scanners**
   Chacun reçoit les mêmes fichiers et renvoie des « findings ». Si un scanner plante, les autres continuent : l'erreur apparaît dans le rapport.

3. **Enlever le bruit**
   Règles désactivées dans `.rempart.toml`, commentaires `rempart:ignore`, doublons.

4. **Appliquer ce qu'il a appris**
   Les faux positifs que tu as marqués disparaissent ; la confiance de chaque règle est recalculée.

5. **Chercher les chemins d'attaque**
   Il combine des résultats de domaines différents en scénarios complets.

6. **Noter et trier**
   Chaque finding reçoit un risque de 0 à 100 ; le dépôt reçoit une note de A à F.

Anatomie d'un finding

CRIT **SQL query built by string interpolation** api/app.py:20

.execute() with an interpolated SQL string (built earlier in `q`).

id 7d9771c7fb · CODE-SQL-INJECTION · risk 81 · static

sévérité — la règle (CWE-89, OWASP A05) — emplacement exact — l'id pour trier

La note du dépôt

A	B	C	D	F
90 et plus	75–89	60–74	40–59	moins de 40

Rempart · triage — app de démo : 1/100

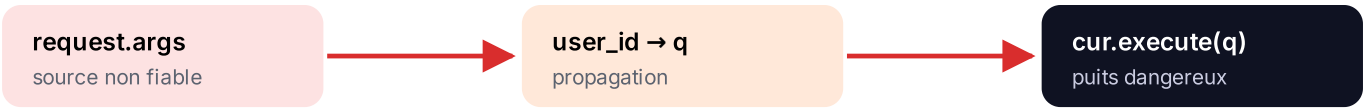
Pourquoi 50 alertes identiques ne coulent pas la note : les répétitions d'une même règle comptent 1, ½, ⅓… Beaucoup de problèmes *différents*, eux, font chuter la note.

EXEMPLE CONCRET 1 · FLUX DE DONNÉES

L'injection SQL cachée dans une variable

Un outil naïf ne regarde que la ligne `execute(q)` : elle a l'air propre. Rempart remonte d'où vient q.

```python
def report(cur):
    # ① vient de l'utilisateur → user_id est « tainted »
    user_id = request.args["id"]
    # ② chaîne construite avec user_id → q est « dynamic »
    q = f"SELECT * FROM t WHERE id = {user_id}"
    # ③ puits SQL + variable marquée → ALERTE
    cur.execute(q)
```



Le correctif proposé

```python
q = "SELECT * FROM t WHERE id = %s"
cur.execute(q, (user_id,))
# la base reçoit la valeur à part : impossible d'injecter du SQL
```

Python

Analyse de l'arbre syntaxique (AST), variable par variable. Chaque fonction du dépôt est résumée, donc la donnée traverse les fonctions et les fichiers (voir page 15).

JavaScript / TS

Suit `req.body`, `request.json()`, `searchParams` et la déstructuration `const { file } = req.query`.

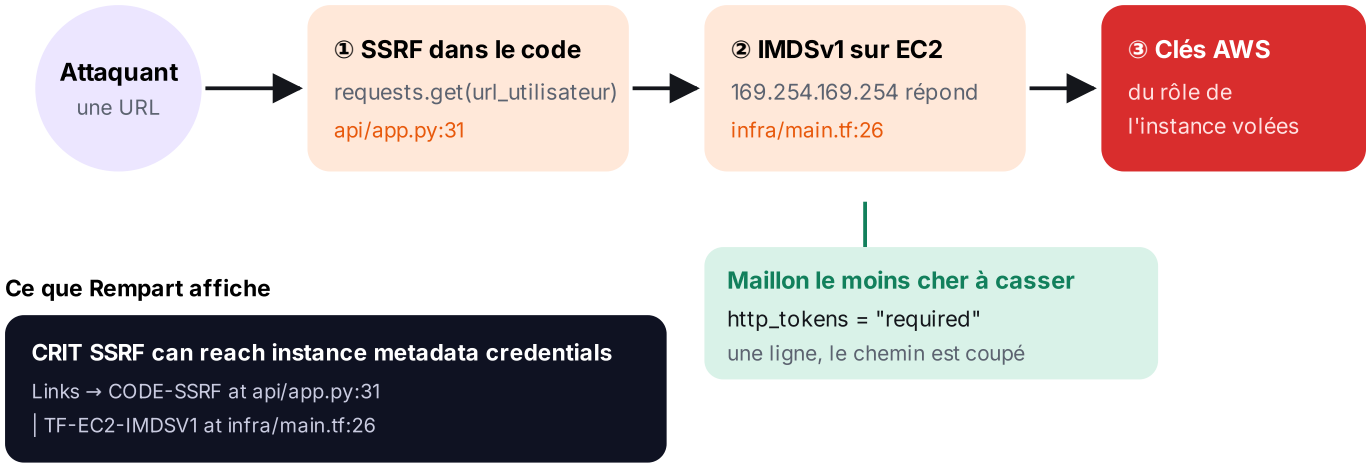
Limite honnête

En JS/TS, le suivi traverse les fonctions d'un même fichier, pas encore les imports entre modules. Là, c'est l'agent IA qui prend le relais.

EXEMPLE CONCRET 2 · CORRÉLATION

Deux « élevés » qui font un « critique »

Pris seuls, une SSRF et IMDSv1 sont deux problèmes moyens. Ensemble, c'est exactement le chemin de la brèche Capital One (2019, 100 millions de dossiers).



Ce que Rempart affiche

```
CRIT SSRF can reach instance metadata credentials
Links → CODE-SSRF at api/app.py:31
| TF-EC2-IMDSV1 at infra/main.tf:26
```

Les 9 scénarios reconnus

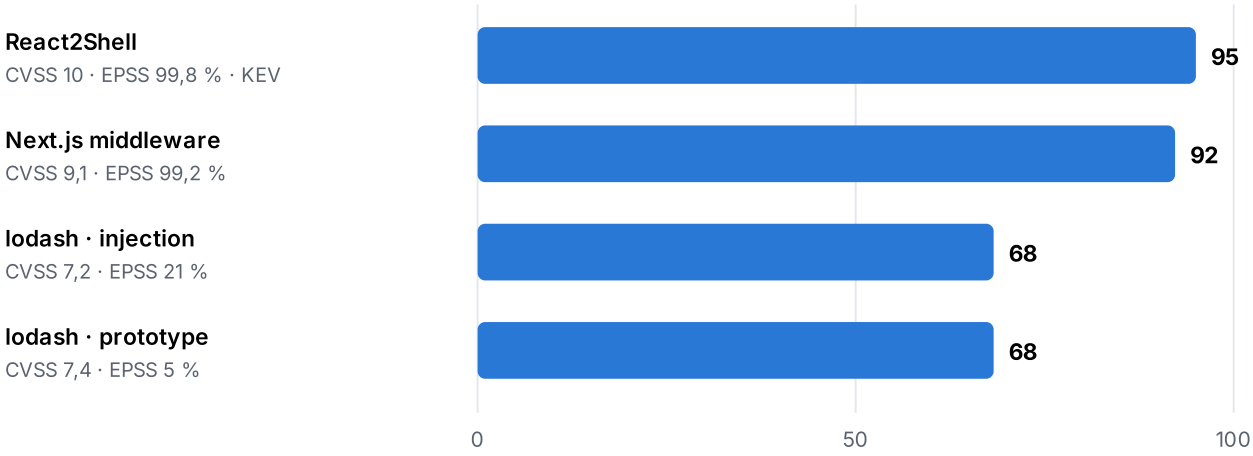
SCÉNARIO	MAILLONS	INCIDENT RÉEL
SSRF → identifiants cloud	SSRF × IMDSv1	Capital One 2019
Base exposée	port public × pas d'auth / pas de chiffrement	rançons MongoDB 2026
CI piégée	code de PR exécuté × droits de publication	TanStack mai 2026
Évasion de conteneur	exécution de code × conteneur privilégié	—
Table publique	RLS manquante × clé publique dans le front	fuites Supabase
Console de debug	debug activé × écoute sur 0.0.0.0	RCE Werkzeug
Vol de sessions	faille XSS × jeton lisible par JavaScript	—
Admin cloud exposé	politique *:* × clé qui peut fuiter	—
Ver npm	code à l'installation × jeton de publication	Shai-Hulud 2025

EXEMPLE CONCRET 3 · PRIORISATION

Toutes les CVE ne se valent pas

Le score CVSS dit « à quel point ce serait grave ». Il ne dit pas « est-ce que quelqu'un l'exploite ». Rempart ajoute deux preuves : la liste KEV de la CISA (exploitée pour de vrai) et EPSS (probabilité d'exploitation dans les 30 jours).

Risque Rempart (0–100) de 4 vraies CVE, mesuré le 9 octobre 2026



La formule, en clair

```
risque = gravité × confiance
si EPSS connu → on le mélange
si dans KEV → au moins 95
```

À retenir

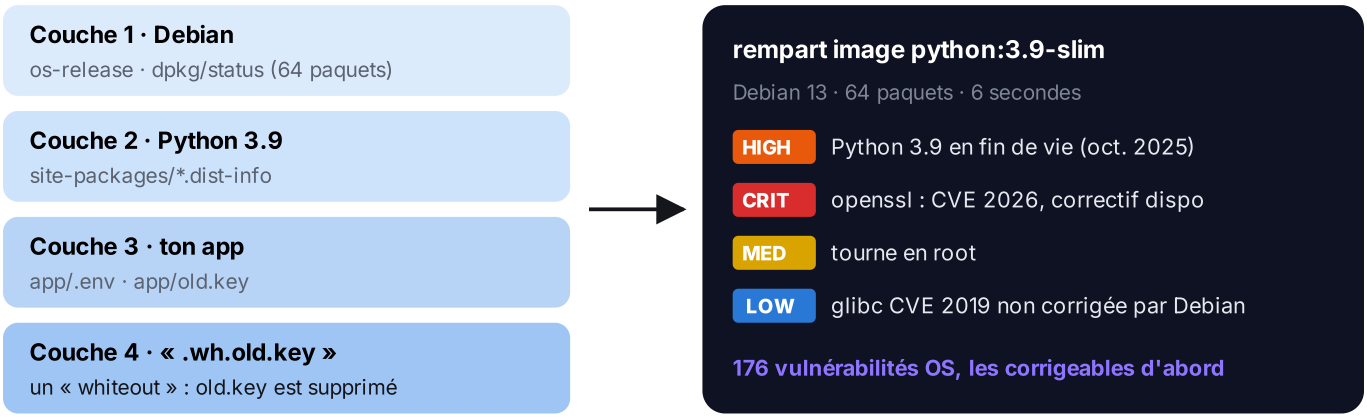
KEV est un **plancher**, pas un bonus : un fait (« c'est exploité ») passe devant une prédiction. C'est la logique de la directive BOD 22-01 de la CISA, la même que dans ton projet triage.

Bonus atteignabilité. Pour chaque paquet vulnérable, Rempart regarde si ton code l'importe vraiment : « imported by your code », « declared », ou « transitive, not imported directly ». Un paquet jamais importé garde son alerte, mais avec une confiance réduite.

EXEMPLE CONCRET 4 · IMAGES

Ouvrir une image Docker… sans Docker

Une image, c'est une pile de couches (des archives .tar). Rempart les télécharge depuis le registre, les superpose, et ne garde que les fichiers utiles.



Fin de vie (EOL)

Alpine 3.18 → fin mai 2025. Une version morte ne reçoit plus de correctifs : les nouvelles CVE resteront ouvertes pour toujours.

Secrets oubliés

Un `.env` copié dans l'image, une variable `API_TOKEN` dans la config ou l'historique de build : quiconque fait `docker pull` peut les lire.

Le tri intelligent

Une CVE de plus de 3 ans que Debian n'a jamais corrigée est souvent contestée ou mineure : gardée, mais classée LOW (sauf si KEV).

En pratique

```
$ rempart image alpine:3.18
alpine:3.18 · Alpine Linux v3.18 · 10 packages · user root · built 2025-02-14
  HIGH  Base OS release is end-of-life      Alpine Linux v3.18 reached end-of-life on 2025-
05-09.
  MEDI  Image runs as root by default       No non-root USER is configured.
  MEDI  Image has not been rebuilt in over a year   Built 602 days ago (2025-02-14).
```

Sortie réelle, image tirée de Docker Hub en 1 seconde. Aucune CVE listée : normal, une version morte ne reçoit plus d'avis de sécurité, c'est justement pour ça que la fin de vie est signalée.

EXEMPLE CONCRET 5 · VÉRIFICATION EN DIRECT

Une clé fuitée : incident ou ménage ?

Une clé déjà révoquée, c'est du nettoyage d'historique. Une clé qui marche encore, c'est un incident. Avec `--verify-secrets`, Rempart pose la question au fournisseur.



24 fournisseurs

AWS (STS) · GitHub · GitLab · Slack · Stripe live · Stripe test · OpenAI · Anthropic · Hugging Face · npm · SendGrid · DigitalOcean · Resend · Linear · Notion · Sentry · Airtable · Figma · Groq · Replicate · Postman · Telegram · Mapbox · New Relic

Le cas AWS

Il faut l'ID de clé **et** le secret à côté, puis signer la requête avec l'algorithme SigV4. Rempart le fait avec la seule bibliothèque standard Python, et un test vérifie que la signature est identique, octet pour octet, à celle de botocore.

Règle d'éthique

Uniquement sur tes propres dépôts. Rien n'est écrit avec la clé, rien n'est stocké : seule l'identité renvoyée (ex. « GitHub user octo ») apparaît.

En pratique

```
$ rempart scan . --verify-secrets
  CRIT  Hard-coded GitHub token  ci.sh:1
        GitHub token found (ghp_**********A5). VERIFIED LIVE: GitHub user octo. Rotate it
now.
        id 3c1e… · SEC-GITHUB-TOKEN · risk 90 · live-secret
```

Exemple tiré des tests automatiques (jeton factice, réponse de GitHub simulée).

EXEMPLE CONCRET 6 · CLOUD EN DIRECT

Ton compte AWS, tel qu'il est vraiment

Terraform dit ce que l'infrastructure *devrait* être. `rempart cloud aws` regarde ce qu'elle *est*, en lecture seule, dans toutes les régions en parallèle.

Identités
MFA du root, clés du root, mots de passe, MFA des utilisateurs, clés de plus de 90 jours, politiques *:*

Stockage
S3 public (ACL, politique, compte), HTTPS non imposé (87 % des comptes AWS en 2026)

Réseau & calcul
Ports SSH/BD ouverts (IPv4 et IPv6), IMDSv1, snapshots et AMI publics, flow logs

Données
RDS public, non chiffré, sans sauvegarde ; chiffrement EBS par défaut ; rotation KMS

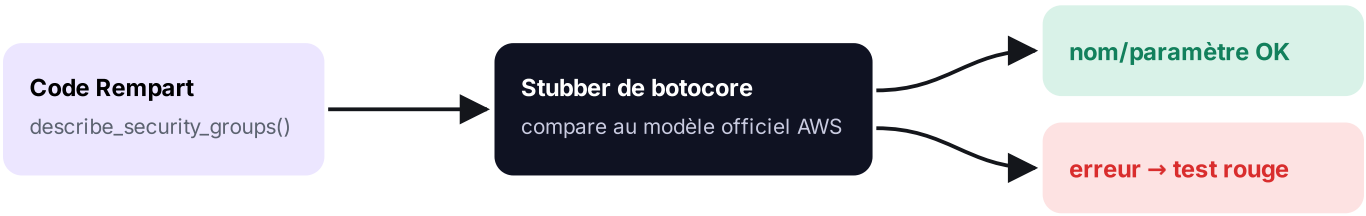
Détection
CloudTrail multi-régions, GuardDuty, AWS Config, Access Analyzer

Serverless
URL Lambda sans auth, runtimes Lambda obsolètes

Kubernetes
Endpoint EKS ouvert à internet

28
contrôles · politique SecurityAudit suffisante

Comment on sait que ça marche sans compte AWS



Preuve que c'est strict : le test a refusé une fausse URL Lambda parce qu'elle faisait 38 caractères au lieu des 40 minimum définis par AWS.

En pratique

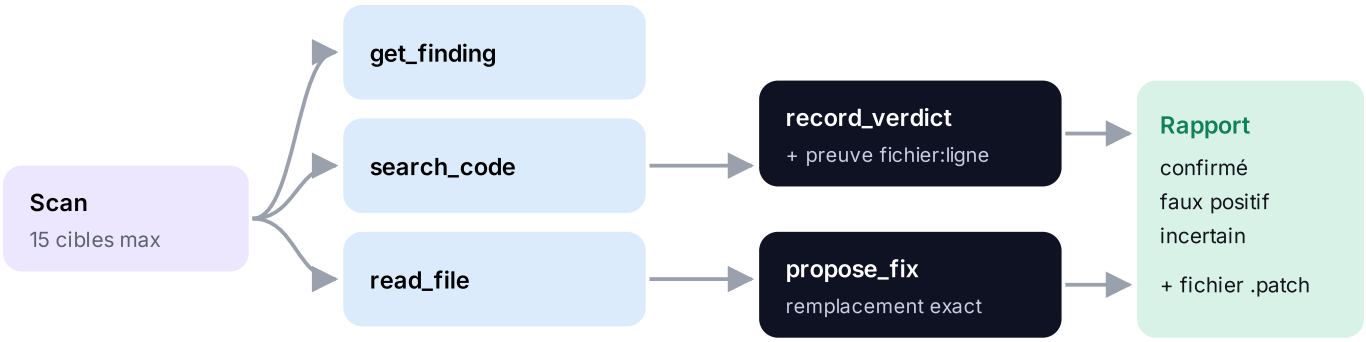
```
$ rempart cloud aws --profile prod
AWS account 123456789012 · all enabled region(s) · read-only
  CRIT  Root account without MFA                        aws://123456789012/iam/root
  CRIT  Security group opens a database port to the internet   …/us-east-1/ec2/sg-1
  HIGH  Running instance allows IMDSv1                  …/us-east-1/ec2/i-1
  …
■ attack-path
  CRIT  SSRF can reach instance metadata credentials   (avec rempart scan . --aws)
```

Forme de sortie produite par les tests (compte simulé 123456789012).

L'AGENT IA

Un relecteur qui lit le code pour toi

Une règle voit un motif. Elle ne sait pas si `eval` reçoit une valeur d'une table constante. L'agent (un modèle d'IA d'Anthropic, famille Opus) va lire, chercher, puis trancher.



l'agent boucle : il lit, cherche, relit… jusqu'à avoir un verdict pour chaque cible (40 tours max)

Exemple de verdict réel (test)

faux positif « eval ne reçoit que des valeurs de la table constante OPS ; l'utilisateur choisit seulement la clé. » `app.py:4,8`

confirmé « request.args['d'] est concaténé dans une commande shell. » `app.py:12`

4 garde-fous

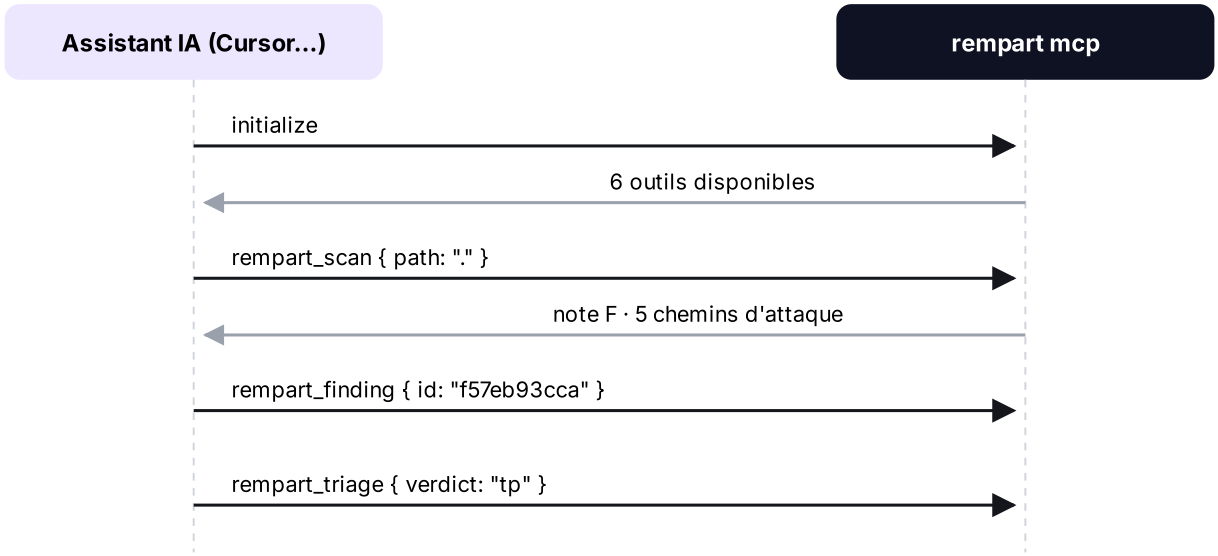
① lecture seule, sur les fichiers du scan uniquement
② le contenu du dépôt est « non fiable » (anti prompt-injection)
③ boucle bornée + repli automatique si le modèle refuse
④ il ne supprime jamais un résultat

Coût affiché à la fin (tokens et dollars estimés). Il envoie des extraits de ton code à l'API Anthropic : à ne pas utiliser sur du code que tu n'as pas le droit de partager.

REMPART DANS TON ÉDITEUR

MCP : ton assistant de code peut l'appeler

MCP (Model Context Protocol) est la prise standard qui permet à un assistant IA d'utiliser des outils. Rempart en est un.



À mettre dans `.mcp.json`

```
{ "mcpServers": { "rempart": { "command": "rempart", "args": ["mcp"] } } }
```

rempart_scan
scanne un dossier

rempart_finding
détail + correctif

rempart_triage
enregistre un verdict

rempart_web_audit
audite un site

rempart_image_scan
analyse une image

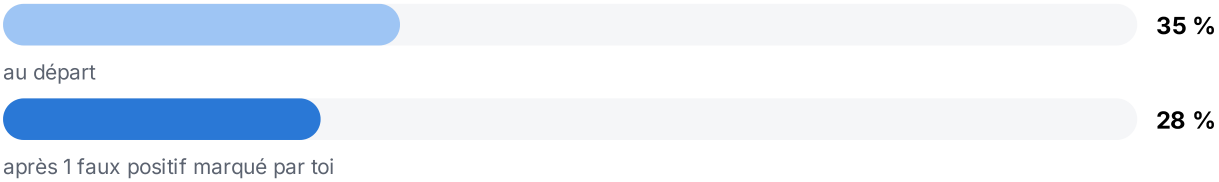
rempart_rules
liste les règles

IL APPREND

Chaque verdict rend l'outil plus précis

Chaque règle a une « confiance de départ ». Tes verdicts la font monter ou descendre. C'est une formule bayésienne simple, mesurée sur un vrai essai.

IDOR (règle volontairement prudente)



IAM « *:* » (règle fiable)




```
rempart triage f26cf13b27 --fp --note "contrôle d'accès dans le middleware"
rempart triage c54d4f3be8 --tp
rempart learn          # ce que l'outil a appris, règle par règle
rempart history        # évolution de la note, nouveaux / corrigés, temps moyen de
correction
```

Toi = décision

Un faux positif que tu marques disparaît des scans suivants de ce projet.

L'IA = indice

Un verdict de l'agent compte 0,5 et ne peut jamais fermer un résultat ni écraser le tien.

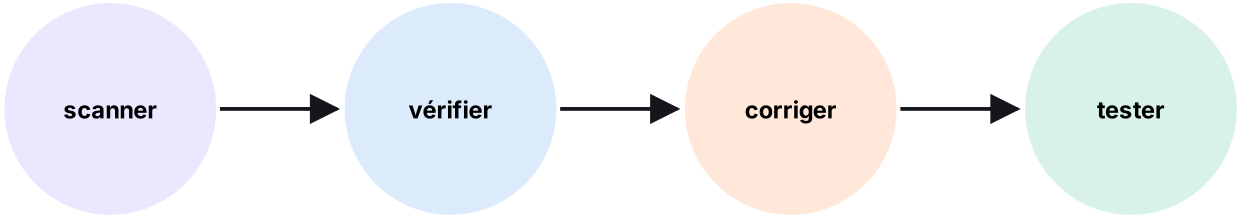
Règle bruyante

5 faux positifs et moins de 25 % de précision : la sévérité baisse d'un cran automatiquement.

LA MÉTHODE

Comment on l'a rendu fiable

Un scanner qui crie au loup tout le temps finit ignoré. La règle suivie : scanner de vrais projets, vérifier chaque alerte à la main, corriger, puis figer le cas dans un test.



CE QUI A ÉTÉ SIGNALÉ	POURQUOI C'ÉTAIT FAUX	LA CORRECTION
`httpOnly: false` sur un cookie (Carriv)	cookie de langue, pas de session	exiger un contexte « session/auth » autour
`{@html tag}` en Svelte (Carriv)	JSON-LD déjà échappé (`\u003c`)	ignorer si le fichier échappe
`/^[^\s.]+(\.[^\s.]+)+$/` « ReDoS »	le point sépare : un seul découpage	analyse des séparateurs
`/Bearer/.exec(header)` « injection »	RegExp.exec ≠ child_process.exec	exiger l'import de child_process
`verdicts.get(x)` « N+1 »	dictionnaire, pas base de données	`.get` seulement sur un ORM

9/9
pièges trouvés dans le labo AWS de cspm-lite (dont 3 manqués au début, corrigés)

A
note de Rempart sur son propre code ; la CI échoue au moindre « high »

0
test sauté en CI : boto3 et le SDK sont installés, tout est exécuté

LE VOCABULAIRE

Glossaire visuel

Les mots que tu entendras en entrevue, en une ligne chacun.

SAST
Analyse statique : lire le code sans l'exécuter.

SCA
Analyse des dépendances : quelles bibliothèques, quelles versions, quelles failles connues.

CVE / CWE
CVE = une faille précise (CVE-2025-55182). CWE = la famille de faille (CWE-89 = injection SQL).

CVSS · EPSS · KEV
Gravité théorique · probabilité d'exploitation · preuve d'exploitation réelle.

Taint (souillure)
Marquer une donnée venant de l'utilisateur et la suivre jusqu'à un endroit dangereux.

SSRF
Faire envoyer au serveur une requête vers une adresse choisie par l'attaquant.

IMDS (v1 / v2)
Service interne d'EC2 qui donne les clés du rôle. v2 exige un jeton : bloque les SSRF simples.

RLS
Row Level Security : la base filtre les lignes selon l'utilisateur (Supabase).

IaC
Infrastructure décrite en code (Terraform) : on peut la scanner avant de la créer.

SARIF
Format standard de résultats de sécurité, affiché par GitHub dans l'onglet Security.

MCP
Protocole qui permet à un assistant IA d'appeler des outils comme Rempart.

EOL
End of life : version qui ne reçoit plus de correctifs de sécurité.

Whiteout
Fichier spécial `.wh.nom` qui supprime un fichier d'une couche d'image précédente.

Faux positif
Alerte sur du code sûr. L'ennemi n°1 d'un scanner.

NOUVEAU EN 0.3 · FLUX DE DONNÉES

Suivre la donnée d'une fonction à l'autre

Avant, le suivi s'arrêtait au bord d'une fonction. Maintenant, Rempart résume **chaque fonction du dépôt** : ce qui entre, ce qui ressort, et ce qui atteint un endroit dangereux.

helpers/separate_request.py

```
def get_form_parameter(self, n):
    return self.request.form.get(n)
def get_safe_value(self, n):
    return "bar"
```

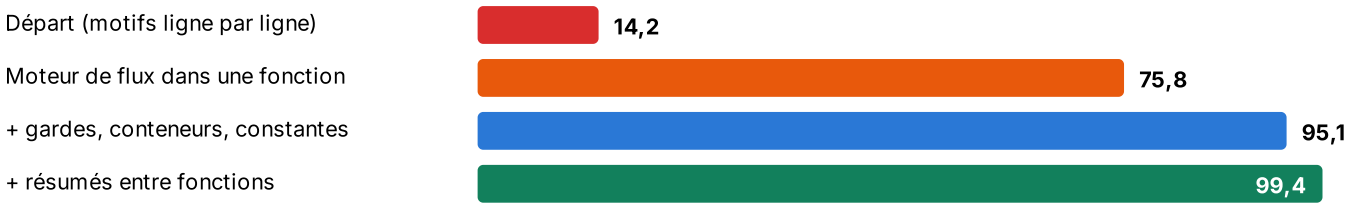
→ **Résumé : renvoie une donnée de la requête**
tout appel → résultat « contaminé »

→ **Résumé : renvoie toujours la constante "bar"**
tout appel → valeur sûre

`param = scr.get_safe_value("x"); eval(param)`	aucune alerte
`param = scr.get_form_parameter("x"); eval(param)`	CRITICAL
`run_tool(request.args["x"]) # le shell est dans un autre fichier`	CRITICAL, avec le fichier et la ligne

Ce que ça a changé, mesuré

Score OWASP Benchmark Python (1 230 cas étiquetés, 0 = hasard, 100 = parfait)

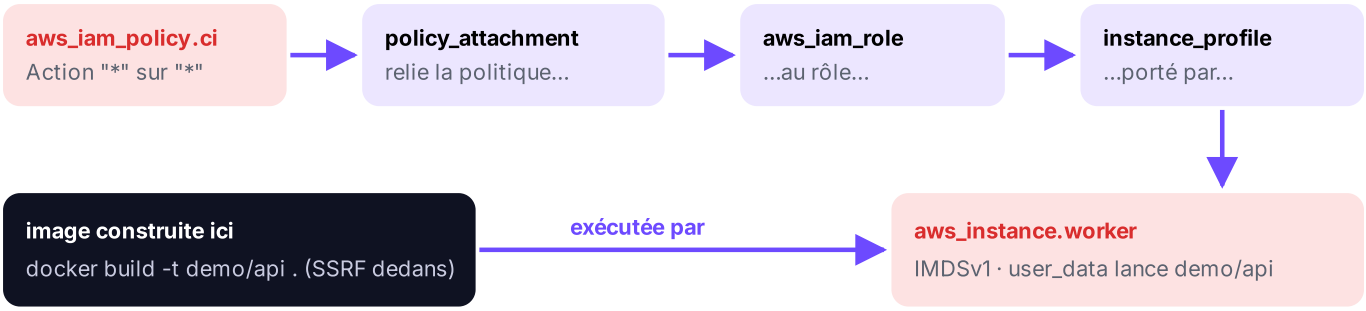


Et en JS/TS ? Les fonctions d'un même fichier sont résumées aussi : `gitLog(req.query.branch)` est signalé si `gitLog` passe son paramètre à `exec`. Un `exec` interpolé sans donnée de requête tracée devient MEDIUM (« à relire »), comme en Python.

NOUVEAU EN 0.3 · GRAPHE DE RESSOURCES

Un chemin d'attaque prouvé, pas supposé

Deux failles dans le même dépôt peuvent concerner deux machines qui ne se parlent pas. Rempart construit maintenant le **graphe** de ce que ton dépôt déploie, et ne dit « lié » que s'il trouve une route entre les deux.



linked: aws_iam_policy.ci → aws_iam_role_policy_attachment.worker_ci → aws_iam_role.worker → aws_iam_instance_profile.worker → aws_instance.worker

Ce qui crée un lien

Une référence Terraform, un depends_on ou un réseau compose, un sélecteur Kubernetes, un compte de service, et l'image que ton dépôt construit, retrouvée dans le déploiement qui la lance.

Lié → CRITICAL

La route s'affiche en vert dans le terminal. C'est elle qu'on montre en revue : on voit la chaîne, maillon par maillon.

Pas de lien → un cran plus bas

« Co-occurrence » : les pièces existent, mais rien ne prouve qu'elles se touchent. Message explicite, sévérité réduite, confiance 0,5.

Sur tes 10 projets : aucun chemin d'attaque n'est sorti, lié ou non. C'est cohérent avec l'étiquetage à la main : aucun ne contenait une vraie chaîne. Un outil qui invente des chemins perd la confiance en une semaine.

NOUVEAU EN 0.3 · AU QUOTIDIEN

Rempart en continu

Un scan ponctuel vieillit le jour même. Quatre outils pour que la sécurité suive le rythme du code.

`--baseline origin/main`

Juger une PR sur ce qu'elle ajoute

Rempart scanne la branche cible dans une copie temporaire (git worktree) et masque tout ce qui existait déjà. Une PR n'hérite plus du passif du dépôt. L'action GitHub le fait par défaut et peut commenter la PR.

`--git-history`

Les secrets qui n'ont jamais vraiment disparu

Effacer une clé d'un fichier ne l'efface pas des commits. Rempart relit chaque ligne ajoutée de chaque branche. **Sur un de tes projets**, il a retrouvé trois clés (OpenAI, Azure Storage, Google) restées dans l'historique d'un `docker-compose.yml`.

`upgrade plan`

Une ligne par paquet, pas 40 alertes

« next 14.2.32 → 15.5.24 : corrige 25 avis sur 25, utilisé par ton code, version majeure ». La plus petite version qui règle tout, en restant sur la même version majeure quand c'est possible. Les dépendances de dev descendent d'un cran.

`rempart watch`

Ne voir que la différence

Rempart surveille le dépôt et rescanne à chaque modification. Il n'affiche que ce qui apparaît (+) ou disparaît (−). Pour la nuit, un workflow planifié relance tout avec les CVE du jour.

Et l'agent IA prouve ce qu'il confirme

```python
# rempart-proofs/test_proof_export.py · écrit par l'agent, jamais exécuté par Rempart
def test_export_escapes_its_folder(client, monkeypatch):
    sent = {}
    fake = lambda p: sent.setdefault("path", p) or ""
    monkeypatch.setattr("files.export.send_file", fake)
    client.get("/export?file=../../etc/passwd")
    # passe aujourd'hui, échoue une fois la faille corrigée
    assert sent["path"].endswith("etc/passwd")
```

Mesurer l'agent : `rempart agent-eval` le fait passer sur 9 petits dépôts étiquetés : 7 vraies failles, 5 pièges (liste blanche dans un helper, nettoyage dans un autre fichier, appelants toujours constants). Il donne le taux d'accord, les vraies failles conservées et les faux positifs écartés.

LA SUITE

Pistes de développement

Classées par horizon. Chaque piste dit ce qu'elle débloque.

1–2 mois

Profondeur

Flux JS/TS entre modules : suivre les imports comme en Python.

Lier le cloud observé au dépôt : rattacher une instance réelle à sa ressource Terraform et à son image.

Chemins confirmés par l'agent, avec test de preuve.

Mesurer l'agent avec une vraie clé et publier le score par modèle.

3–6 mois

Largeur

Plus de contrôles GCP et Azure (journaux, KMS, GKE, AKS).

RPM ancien format (RHEL 7/8) et flux d'avis Red Hat.

Kubernetes : politiques d'admission, NetworkPolicy.

Politique « cooldown » : refuser un paquet publié il y a moins de N jours.

6–12 mois

Produit

Publication PyPI : le workflow est prêt, il suffit d'un tag `v0.3.0`.

Image Docker de Rempart, minimale et non-root.

Tableau de bord web multi-projets avec historique.

Version équipe : calibration partagée, intégration Slack, SSO.

Pour ta carrière : chaque piste est un sujet d'entrevue. Le graphe de ressources parle aux équipes cloud, le flux entre modules aux équipes AppSec, le tableau de bord aux équipes produit.

AIDE-MÉMOIRE

Les commandes à connaître

Garde cette page sous la main.

COMMANDE	CE QU'ELLE FAIT
`rempart scan .`	scanne le dépôt courant
`rempart scan . -f html -o rapport.html`	rapport HTML filtrable
`rempart scan . --fail-on high`	code de sortie 1 s'il y a du « high » (CI)
`rempart scan . --aws --image app:latest`	code + AWS + image, corrélés
`rempart scan . --verify-secrets`	teste si les clés trouvées marchent encore
`rempart image python:3.12-slim`	analyse une image sans Docker
`rempart cloud aws --profile prod`	audit AWS en direct
`rempart cloud gcp` · `rempart cloud azure`	audit GCP / Azure en direct
`rempart scan . --baseline origin/main`	seulement ce que la branche ajoute
`rempart scan . --git-history`	secrets restés dans l'historique
`rempart watch .`	rescan à chaque modification
`rempart agent-eval`	mesurer l'agent IA
`rempart k8s --context prod`	audit du cluster
`rempart web https://carriv.com`	audit passif de ton site
`rempart agent .`	l'IA vérifie et propose des correctifs
`rempart triage <id> --fp` · `history`	apprendre à l'outil (--fp, --tp, --accept) · ta progression
`rempart mcp`	serveur pour assistants de code IA
`rempart rules --domain data`	liste des règles d'un domaine

Ignorer une alerte précise

```
eval(x)  # rempart:ignore[CODE-EVAL]
```

Configurer un projet

```
.rempart.toml
exclude = ["fixtures/**"]
fail_on = "high"
```